

Uniwersytet Wrocławski

Generatory Liczb Pseudolosowych

Autorzy: Łukasz Stodółka, Łukasz Besuch, Marcin Rypula

Wrocław, 2026

Contents

1	What is a Pseudorandom Number Generator?	3
1.1	Formal Definition	3
1.2	Seed (Initial Value)	3
1.3	State Space and Periodicity	3
1.4	Uniformity and Equidistribution	4
1.5	Computational Randomness	4
1.6	Summary	4
2	Classical Methods of Pseudorandom Number Generation	6
2.1	Linear Congruential Generator (LCG)	6
2.1.1	Period of LCG	6
2.1.2	Properties and Limitations	7
2.1.3	Example of LCG	7
2.2	Lehmer Generator	7
2.2.1	Period	7
2.2.2	Properties	8
2.3	Combined Generators	8
2.3.1	Period	8
2.3.2	Advantages	8
2.4	Summary	8
3	Recursive Generators: LFSR and LFG	9
3.1	Linear Feedback Shift Register (LFSR)	9
3.2	The Mathematics of LFSR	9
3.2.1	The Recurrence Relation	9
3.2.2	Polynomial Representation and State Shifts	9
3.2.3	Algebraic Structure and Finite Fields	10
3.2.4	The Role of the Least Common Multiple (LCM)	11
3.2.5	Periodicity and Minimal Polynomials	11
3.3	Example: 4-Bit LFSR	12
4	The Mersenne Twister: High-Dimensional Equidistribution	13
4.1	The Twisted Generalized Feedback Shift Register (TGFSR)	13
4.2	The Twist	13
4.3	Tempering: Guaranteeing Equidistribution	14
4.4	Periodicity and Mersenne Primes	14
4.5	Variants of the Mersenne Twister	15
4.5.1	The MT19937 Variant	15

4.5.2	Other variants of the MT	15
5	Conclusion and Comparison of PRNG Methods	17
5.1	General Comparison Criteria	17
5.2	Classical Generators	17
5.3	Recursive and Shift-Based Generators	18
5.4	Modern High-Quality Generators	18
5.5	Trade-offs Between Methods	18
5.6	Final Remarks	18

1 What is a Pseudorandom Number Generator?

A pseudorandom number generator (PRNG) is an algorithm that produces a sequence of numbers that approximates the properties of truly random sequences.

Unlike true randomness, which is based on physical and inherently unpredictable processes, a PRNG is fully deterministic. This means that if the initial state (seed) is known, the entire sequence can be reproduced exactly.

1.1 Formal Definition

Formally, a PRNG can be described as a discrete dynamical system consisting of:

- a finite state space S ,
- a deterministic transition function $f : S \rightarrow S$,
- an output function $g : S \rightarrow \mathbb{R}$.

The generator is defined by the recurrence relation:

$$x_{n+1} = f(x_n),$$

with initial state (seed) $x_0 \in S$, and output sequence:

$$u_n = g(x_n).$$

Here, (x_n) represents the internal state sequence, while (u_n) represents the generated pseudorandom numbers.

1.2 Seed (Initial Value)

The seed x_0 is the initial element of the state space S . It uniquely determines the entire sequence generated by the PRNG.

If the same seed is used multiple times, the generator produces the same sequence, which is a key property used for reproducibility in simulations, debugging, and numerical experiments.

Different seeds lead to different trajectories in the state space, increasing variability of outputs.

1.3 State Space and Periodicity

Since the state space S is finite and the generator is deterministic, the sequence (x_n) must eventually repeat. This follows from the pigeonhole principle: after a finite number of steps, some state must reoccur, causing the sequence to enter a cycle.

Thus, every PRNG becomes eventually periodic.

The period of a PRNG is the smallest positive integer T such that:

$$x_{n+T} = x_n \quad \text{for all } n \geq n_0,$$

for some transient length $n_0 \geq 0$.

A good PRNG is characterized by a very long period, making repetition practically unobservable in applications.

1.4 Uniformity and Equidistribution

Uniformity refers to how evenly the output values are distributed over a given interval.

For normalized outputs $u_n \in [0, 1]$, a sequence is considered uniformly distributed if, for any subinterval $[a, b] \subset [0, 1]$, we have:

$$\lim_{N \rightarrow \infty} \frac{1}{N} \# \{1 \leq n \leq N : u_n \in [a, b]\} = b - a.$$

This property is also known as *equidistribution*.

Lack of uniformity introduces bias and can significantly distort results in simulations and statistical applications, such as Monte Carlo methods.

1.5 Computational Randomness

Although PRNGs are deterministic, their goal is to produce sequences that are statistically indistinguishable from truly random sequences under practical tests.

In other words, while they are not random in a physical sense, they exhibit *computational randomness*, meaning that without knowledge of the seed, the sequence appears random for algorithmic and statistical purposes.

1.6 Summary

A pseudorandom number generator is a deterministic system defined on a finite state space that produces sequences with long periods and strong statistical properties.

A good PRNG should satisfy:

- determinism (for reproducibility),
- long period,
- efficient computation,
- good statistical properties such as uniformity and equidistribution.

Many classical PRNGs, such as linear congruential generators, Lehmer generators, and LFSR-based systems, are specific implementations of this general framework, defined by particular choices of the transition function f and output function g .

2 Classical Methods of Pseudorandom Number Generation

Classical pseudorandom number generators are based on simple mathematical constructions, typically involving modular arithmetic and linear recurrence relations. Despite their simplicity, they form the basis for many modern designs and provide important insight into the structure of deterministic random-like sequences.

In this section, we present three fundamental classes of classical generators: the Linear Congruential Generator (LCG), the Lehmer generator, and combined generators.

2.1 Linear Congruential Generator (LCG)

The Linear Congruential Generator is one of the simplest and most widely studied pseudorandom number generators. It is defined by the recurrence relation:

$$x_{n+1} = (ax_n + c) \mod m,$$

where:

- x_n is the current state,
- a is the multiplier,
- c is the increment,
- m is the modulus,
- x_0 is the seed.

The state space of an LCG is the finite ring $\mathbb{Z}_m$, and thus all values satisfy $x_n \in \{0, 1, \dots, m-1\}$.

2.1.1 Period of LCG

The maximum possible period of an LCG is m . This full period is achieved if and only if the following conditions hold:

- $\gcd(c, m) = 1$,
- $a - 1$ is divisible by all prime factors of m ,
- if m is divisible by 4, then $a - 1$ is also divisible by 4.

These conditions are known as the Hull–Dobell theorem, which characterizes full-period LCGs.

2.1.2 Properties and Limitations

LCGs are computationally efficient and easy to implement. However, their statistical quality depends strongly on the choice of parameters.

A well-known drawback is that sequences generated by LCGs exhibit lattice structure when considered in higher dimensions. This geometric regularity reduces their suitability for high-dimensional simulations such as Monte Carlo methods.

Additionally, the period may be significantly smaller than m if parameters are poorly chosen.

2.1.3 Example of LCG

Example with parameters: $m = 9$, $a = 2$, $c = 1$, $x_0 = 1$:

n	x_n
0	1
1	3
2	7
3	6
4	4
5	0
6	1

Table 1: Example sequence generated by an LCG

2.2 Lehmer Generator

The Lehmer generator is a special case of the LCG where the increment $c = 0$. It is defined by:

$$x_{n+1} = (ax_n) \mod m.$$

It is also known as the multiplicative congruential generator.

In this case, the state evolves in the multiplicative group $\mathbb{Z}_m$.

2.2.1 Period

To achieve a long period, the modulus m is typically chosen as a prime number, and the multiplier a is chosen as a primitive root modulo m .

Under these conditions, the generator achieves the maximum period:

$$T = m - 1.$$

2.2.2 Properties

- simpler structure compared to full LCG,
- good statistical properties for appropriate parameter choices,
- still vulnerable to correlation effects and structural artifacts.

2.3 Combined Generators

Combined generators are constructed by combining multiple simpler generators in order to improve statistical properties and increase the effective period.

A typical construction is:

$$x_n = (x_n^{(1)} - x_n^{(2)}) \mod m,$$

where $x_n^{(1)}$ and $x_n^{(2)}$ are independent sequences generated by two separate generators.

2.3.1 Period

If the individual generators have periods T_1 and T_2 , then the period of the combined generator satisfies:

$$T \mid \text{lcm}(T_1, T_2),$$

meaning that T divides the least common multiple of the component periods.

2.3.2 Advantages

- increased effective period,
- improved statistical behavior compared to single generators,
- reduced correlation between successive outputs.

2.4 Summary

Classical pseudorandom number generators such as LCG and Lehmer generators are simple and efficient but exhibit structural weaknesses, particularly in higher dimensions.

Combined generators partially address these issues by increasing period length and reducing correlation effects, but they still inherit linearity-based limitations.

These classical methods provide the foundation for understanding more advanced pseudorandom number generators introduced in later chapters.

3 Recursive Generators: LFSR and LFG

3.1 Linear Feedback Shift Register (LFSR)

It is incredibly fast, operates fundamentally on binary numbers (0s and 1s), and is widely used in digital electronics and simple cryptography. LFSR consists of:

1. **Shift Register:** A device that shifts contents into adjacent positions in register or out of register if position is on the end. The positions where the content was is left empty if there is no content that that could be shifted there.
2. **Output Bit:** The bit at the end that is moved out of a shift register.
3. **Feedback Function:** A function that takes bits from certain positions "**taps**", combines them in some sort of function and the result is fed back into the register's input bit. Bits are collected before the shift.

An n -bit LFSR has 2^n possible states. However, because a state of all zeros will infinitely produce zeros, the maximum possible period for an LFSR sequence is $2^n - 1$.

3.2 The Mathematics of LFSR

3.2.1 The Recurrence Relation

The new bit b_n in a Linear Feedback Shift Register (LFSR) is calculated recursively based on the previous bits. Given a register of length k with feedback taps defined by coefficients c_i (where $c_i = 1$ if there is a active tap, and 0 if there is not), the sequence generation is governed by the following formula:

$$b_n = (c_1b_{n-1} + c_2b_{n-2} + \cdots + c_kb_{n-k}) \pmod{2}$$

(Note: In binary mathematics, addition modulo 2 is equivalent to the logical XOR operation).

3.2.2 Polynomial Representation and State Shifts

This feedback relationship can be elegantly modeled using polynomials over the binary field. Let us define the characteristic polynomial p and its truncated variant $\tilde{p}$ as:

$$p = x^{n+1} + b_nx^n + \cdots + b_1x + b_0$$

$$\tilde{p} = b_nx^n + \cdots + b_1x + b_0$$

If the current state of the register is represented as a polynomial q :

$$q = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0$$

A single shift of the register corresponds to multiplying the state polynomial by the indeterminate x :

$$x \cdot q = a_n x^{n+1} + a_{n-1} x^n + \cdots + a_0 x$$

To keep the state within the bounds of the register, we reduce this result modulo the characteristic polynomial p :

$$xq \pmod{p} = xq - a_n p$$

Expanding this modular reduction yields the polynomial configuration for the subsequent state:

$$(a_{n-1} - b_n)x^n + (a_{n-2} - b_{n-1})x^{n-1} + \cdots + (a_0 - b_1)x - b_0$$

3.2.3 Algebraic Structure and Finite Fields

To analyze the periodicity and structural properties of the generated sequences, we look at the underlying algebraic frameworks. By the Chinese Remainder Theorem for integers, we know that if m and n are coprime, there exists an isomorphic mapping:

$$\mathbb{Z}_{mn} \rightarrow \mathbb{Z}_m \times \mathbb{Z}_n$$

$$k \rightarrow (k \bmod m, k \bmod n)$$

Because m and n are coprime ($\gcd(m, n) = 1$), this representation is a one-to-one mapping (an injection, and by cardinality, a bijection). An analogous property holds for the polynomial ring over a field K :

$$K[x]/(p_1 \cdot p_2) \cong K[x]/(p_1) \times K[x]/(p_2)$$

If the characteristic polynomial p is irreducible over $K = \mathbb{Z}_2$, the quotient ring forms a Galois finite field $F = K[x]/(p)$ containing exactly 2^n elements. For any non-zero element $y \in F \setminus \{0\}$, by Lagrange's theorem, its multiplicative order must divide the order of the group, meaning:

$$y^k \cdot y^{2^n-1} = 1$$

$$y^{2^n-1} - 1 = 0 \implies \forall (y \in F \setminus \{0\}), y^{2^n} - y = 0$$

3.2.4 The Role of the Least Common Multiple (LCM)

When analyzing elements in a direct product of structures, such as $y \in \mathbb{Z}_{t_1} \times \mathbb{Z}_{t_2}$, the periodicity of the combined system is determined by the *Least Common Multiple* (lcm).

Mathematically, if an element y_1 has a period of t_1 (meaning $y_1^{t_1} = 1$) and an element y_2 has a period of t_2 (meaning $y_2^{t_2} = 1$), then the joint element $y = (y_1, y_2)$ will only return to the identity state when both coordinates simultaneously complete full cycles. This global period is the smallest positive integer L that is a multiple of both t_1 and t_2 :

$$y^{\text{lcm}(t_1, t_2)} = (1, 1)$$

If t_1 and t_2 share common factors, $\text{lcm}(t_1, t_2)$ is strictly less than their product $t_1 \cdot t_2$, shrinking the total period of the system. Maximum sequence length is only achieved when components operate at coprime periods, pushing the lcm to its absolute maximum upper bound: $t_1 \cdot t_2$.

3.2.5 Periodicity and Minimal Polynomials

To determine if an LFSR sequence can achieve maximum length, we investigate if there exists a smaller exponent $l < 2^n - 1$ such that $y^l = 1$ for all elements. If $\forall (y \in F), y^{l+1} - y = 0$, it implies that $l + 1 \geq 2^{n-1}$.

Because the multiplicative group of the finite field F is cyclic, there exists a primitive generator element $y \in F$ whose successive powers y^k for $k = 0, \dots, 2^n - 2$ are completely distinct and map out all non-zero elements. Let h be the minimal polynomial of y . By definition, $h(y) = 0$, h is monic, and its degree is minimal among all polynomials g satisfying $g(y) = 0$.

Consequently, h must be irreducible. If there exists some polynomial g such that $g(y) = 0$, applying the polynomial division algorithm yields:

$$g = qh + r \quad \text{where } \deg(r) < \deg(h)$$

Evaluating this at y gives $g(y) = q(y)h(y) + r(y) \implies 0 = 0 + r(y)$. Since $r(y) = 0$ and its degree is strictly less than the minimal polynomial h , it must hold that $r = 0$, proving that g is an exact multiple of h ($g = qh$).

The evaluation mapping $Y \rightarrow y$ establishes an isomorphism from the quotient ring $K[Y]/(h[Y])$ to the finite field F . Most importantly, sequential powers of Y generate every non-zero element of $K[Y]/(h[Y])$.

Conclusion: To achieve the maximum possible period of $2^n - 1$ states without stalling or repeating early, the characteristic polynomial $P(x)$ must be a **primitive polynomial** over $GF(2)$.

3.3 Example: 4-Bit LFSR

Consider a 4-bit LFSR with a tap sequence at positions 3 and 4, utilizing XOR as the feedback function.

a	b	c(tap)	d(tap)	Output
1	0	0	1	1
1	1	0	0	0
0	1	1	0	0
1	0	1	1	1
0	1	0	1	1
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1
0	1	1	1	1
0	0	1	1	1
0	0	0	1	1
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
1	0	0	1	1

Table 2: Example of 4-bit LFSR with taps c, b

4 The Mersenne Twister: High-Dimensional Equidistribution

Developed in 1997 by Makoto Matsumoto and Takuji Nishimura, the Mersenne Twister (MT) is a widely used pseudorandom number generator in modern computing, serving as the default or standard option in software environments including Python's `random` module, R, MATLAB, and the C++ standard library.

By choosing parameters such that the effective state size $nw - r$ corresponds to a Mersenne exponent, the MT algorithm achieves a staggering periodicity and an extraordinarily high degree of multidimensional equidistribution.

4.1 The Twisted Generalized Feedback Shift Register (TGFSR)

The Mersenne Twister, is based on Generalized Feedback Shift Register (GFSR). While a standard LFSR operates on single bits over $\mathbb{F}_2$, a GFSR operates on w -bit words, effectively running w identical LFSRs in parallel.

The Mersenne Twister is a specific optimization of a *Twisted* Generalized Feedback Shift Register (TGFSR). Let the word size be w bits, and let the state array consist of n words. The state at step k is represented by a sequence of w -bit row vectors $\mathbf{x}_k \in \mathbb{F}_2^w$.

4.2 The Twist

The TGFSR defines the state transition via a linear recurrence relation of order n . The generator computes the subsequent word $\mathbf{x}_{k+n}$ using the word from n steps ago ($\mathbf{x}_k$) and a word from a mid-point m steps ago ($\mathbf{x}_{k+m}$), augmented by a linear transformation matrix A :

$$\mathbf{x}_{k+n} = \mathbf{x}_{k+m} \oplus \left(\mathbf{x}_k^u \mid \mathbf{x}_{k+1}^l \right) A$$

Let us rigorously dissect the components of this linear transformation over $\mathbb{F}_2$:

Concatenation ($\mid$): The term $\left(\mathbf{x}_k^u \mid \mathbf{x}_{k+1}^l \right)$ represents a bitwise concatenation. We extract the upper $w - r$ bits from $\mathbf{x}_k$ (denoted $\mathbf{x}_k^u$) and the lower r bits from the next state $\mathbf{x}_{k+1}$ (denoted $\mathbf{x}_{k+1}^l$). This creates a new, composite w -bit word. This incomplete word transition is one of the key innovations of the Mersenne Twister architecture, allowing the state space to precisely match a Mersenne prime rather than a power of 2.

The Matrix A : The concatenated word is multiplied by a $w \times w$ matrix A over $\mathbb{F}_2$. To ensure maximum computational efficiency in software, A is chosen to be in Rational

Normal Form (a companion matrix):

$$A = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ a_{w-1} & a_{w-2} & a_{w-3} & \cdots & a_0 \end{pmatrix}$$

Remark. Because A is in rational normal form, multiplying a vector $\mathbf{y} = (y_{w-1}, \dots, y_0)$ by A can be executed via a single bit-shift and a conditional XOR. If the lowest bit $y_0 = 0$, the result is $\mathbf{y} \gg 1$. If $y_0 = 1$, the result is $(\mathbf{y} \gg 1) \oplus \mathbf{a}$, where $\mathbf{a} = (a_{w-1}, \dots, a_0)$.

4.3 Tempering: Guaranteeing Equidistribution

To avoid high degrees of linear correlation, failing rigorous tests for k -distribution, we add **tempering** to the process, which is an invertible linear transformation over $\mathbb{F}_2$. The tempering function $T : \mathbb{F}_2^w \rightarrow \mathbb{F}_2^w$ is designed to multiply the state vector by an invertible matrix T , which systematically scrambles the bits to achieve an optimal multidimensional distribution.

Given a raw state word $\mathbf{x}$, the tempered output $\mathbf{y}$ is generated through four successive shift-and-mask operations, utilizing shifting constants (u, s, t, l) and bitmask vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}$:

$$\begin{aligned} \mathbf{y}_1 &= \mathbf{x} \oplus ((\mathbf{x} \gg u) \& \mathbf{a}) \\ \mathbf{y}_2 &= \mathbf{y}_1 \oplus ((\mathbf{y}_1 \ll s) \& \mathbf{b}) \\ \mathbf{y}_3 &= \mathbf{y}_2 \oplus ((\mathbf{y}_2 \ll t) \& \mathbf{c}) \\ \mathbf{y} &= \mathbf{y}_3 \oplus ((\mathbf{y}_3 \gg l) \& \mathbf{d}) \end{aligned}$$

where $\&$ denotes the bitwise AND operation.

Twierdzenie 4.1. Because each step of the tempering process is an invertible linear transformation over $\mathbb{F}_2$, the composition of these transformations is likewise invertible. Hence, the entire tempering process defines a bijection on $\mathbb{F}_2^w$.

4.4 Periodicity and Mersenne Primes

The defining characteristic of the Mersenne Twister is its periodicity. The total number of bits in the internal state is $n \times w$. However, because r bits are masked out during the concatenation step $(\mathbf{x}_k^u \mid \mathbf{x}_{k+1}^l)$, the actual operational state space consists of $nw - r$ bits.

Therefore, the maximum possible period of the generator is $2^{nw-r} - 1$.

Definicja 4.2. A *Mersenne prime* is a prime number that can be written in the form $M_p = 2^p - 1$, where p is a prime integer.

By carefully choosing the parameters n , w , and r such that $p = nw - r$ corresponds to the exponent of a Mersenne prime, Matsumoto and Nishimura simplified the mathematical analysis required to construct a primitive characteristic polynomial for the generator.

4.5 Variants of the Mersenne Twister

4.5.1 The MT19937 Variant

The most popular configuration of the Mersenne Twister. The parameters are chosen as follows:

- Word size $w = 32$ bits.
- Degree of recurrence $n = 624$ words.
- Separation point $m = 397$.
- Lower bitmask size $r = 31$ bits.

This configuration results in a total operational state size of $624 \times 32 - 31 = 19937$ bits. The parameters of MT19937 were specifically selected so that the associated characteristic polynomial is primitive over $\mathbb{F}_2$, yielding the maximal possible period

$$2^{19937} - 1.$$

The fact that 19937 is a Mersenne exponent plays an important role in the underlying construction and analysis.

4.5.2 Other variants of the MT

1. **CryptMT:** CryptMT is a stream cipher and cryptographically secure pseudorandom number generator (CSPRNG) It was developed by Makoto Matsumoto and Takuji Nishimura alongside Mariko Hagita and Mutsuo Saito. Unlike the classical Mersenne Twister or its other structural derivatives, CryptMT is patented.
2. **MTGP:** MTGP (Mersenne Twister for Graphics Processors) is a variant optimized for graphics processing units published by Mutsuo Saito and Makoto Matsumoto. The basic linear recurrence operations are extended from MT, and parameters are carefully chosen to allow a massive pool of parallel threads to compute the recursion concurrently while sharing their active state space to minimize memory load.

The original paper claims improved multidimensional equidistribution over standard MT, displaying execution metrics on a 2008-era GPU (Nvidia GTX260 with 192 cores) of 4.7 ms to produce 5×10^7 random 32-bit integers.

3. **SFMT:** SFMT (SIMD-oriented Fast Mersenne Twister) is a variant introduced in 2006 designed specifically to leverage speed advantages when executing on 128-bit SIMD architectures. It is roughly twice as fast as the classical Mersenne Twister.
4. **TinyMT:** TinyMT is a lightweight variant proposed by Saito and Matsumoto in 2011. It utilizes just 127 bits of internal state space, marking a significant footprint reduction compared to the 2.5 KiB state array required by the classic MT19937. However, its maximum period is bounded at:

$$2^{127} - 1$$

5 Conclusion and Comparison of PRNG Methods

This section summarizes and compares the pseudorandom number generators discussed in the previous chapters. The goal is to highlight their main differences in terms of period length, computational efficiency, statistical quality, and structural complexity.

5.1 General Comparison Criteria

Pseudorandom number generators can be evaluated according to several fundamental criteria:

- **Period length:** the length of the cycle before the sequence repeats.
- **Computational efficiency:** the cost of generating a single value.
- **Statistical quality:** how well the output approximates a truly random sequence.
- **Structural complexity:** mathematical and implementation complexity of the generator.

These criteria are often in conflict; for example, higher statistical quality typically comes at the cost of increased complexity.

5.2 Classical Generators

Classical methods such as linear congruential generators (LCG) and Lehmer generators are based on simple modular arithmetic:

$$x_{n+1} = (ax_n + c) \mod m$$

or in the multiplicative case:

$$x_{n+1} = ax_n \mod m.$$

These generators are computationally very efficient and easy to implement. However, their statistical properties depend strongly on parameter choice.

The period of an LCG is at most m , and full period is achieved only under the Hull–Dobell conditions.

A known drawback is their tendency to exhibit geometric structure when viewed in higher dimensions, which reduces their suitability for high-quality simulations.

5.3 Recursive and Shift-Based Generators

More advanced recursive methods such as lagged Fibonacci generators (LFG) and linear feedback shift registers (LFSR) improve statistical properties by introducing longer memory into the state:

- LFG uses previous values in a recurrence relation.
- LFSR operates on bit-level linear recurrences over finite fields.

These methods generally achieve longer periods and better distribution properties than simple LCGs, while maintaining moderate computational cost.

5.4 Modern High-Quality Generators

Modern generators such as the Mersenne Twister, PCG, and xorshift-based methods are designed to optimize statistical quality and period length.

The Mersenne Twister, for example, achieves an extremely large period of $2^{19937} - 1$, making repetition practically impossible in most applications.

PCG (Permuted Congruential Generator) improves statistical behavior by applying permutation functions to linear congruential states, while maintaining efficiency.

Xorshift generators rely on bitwise XOR and shift operations, providing extremely fast generation with good statistical performance when properly parameterized.

5.5 Trade-offs Between Methods

There is no single optimal PRNG for all applications. Instead, different methods optimize different aspects:

- **LCG / Lehmer:** very fast, but weaker statistical quality.
- **LFSR / LFG:** better structure and longer period, moderate complexity.
- **Mersenne Twister:** excellent statistical properties, very long period, higher state complexity.
- **PCG / Xorshift:** modern balance between speed and quality.

5.6 Final Remarks

Pseudorandom number generation is fundamentally a trade-off between simplicity, speed, and statistical quality.

While classical methods such as LCG provide insight into the structure of deterministic sequences, modern generators achieve practical randomness suitable for large-scale simulations and computational applications.

Understanding the underlying mathematical structure of these generators is essential for selecting appropriate methods in scientific computing, cryptography, and numerical simulations.

References

- [1] Donald Knuth. “The Art of Computer Programming”. In: (1997).

test